

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
```

```
df=pd.read_csv("dailyActivity_merged.csv")
df.head()
```

	TotalSteps	TotalDistance	VeryActiveDistance	ModeratelyActiveDistance	LightActiveDistance	VeryActiveMinutes	FairlyActiveMinutes
0	13162	8.50	1.88	0.55	6.06	25	
1	10735	6.97	1.57	0.69	4.71	21	
2	10460	6.74	2.44	0.40	3.91	30	
3	9762	6.28	2.14	1.26	2.83	29	
4	12669	8.16	2.71	0.41	5.04	36	

```
print("Shape of dataset:", df.shape)
print("\nData Info:\n")
df.info()
```

```
print("\nMissing Values:\n")
print(df.isnull().sum())
```

Shape of dataset: (599, 10)

Data Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 599 entries, 0 to 598
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   TotalSteps                            599 non-null    int64
1   TotalDistance                         599 non-null    float64
2   VeryActiveDistance                   599 non-null    float64
3   ModeratelyActiveDistance             599 non-null    float64
4   LightActiveDistance                  599 non-null    float64
5   VeryActiveMinutes                    599 non-null    int64
6   FairlyActiveMinutes                  599 non-null    int64
7   LightlyActiveMinutes                 599 non-null    int64
8   SedentaryMinutes                     599 non-null    int64
9   Calories                             599 non-null    int64
dtypes: float64(4), int64(6)
memory usage: 46.9 KB
```

Missing Values:

```
TotalSteps          0
TotalDistance        0
VeryActiveDistance   0
ModeratelyActiveDistance  0
LightActiveDistance  0
VeryActiveMinutes    0
FairlyActiveMinutes  0
LightlyActiveMinutes 0
SedentaryMinutes     0
Calories             0
dtype: int64
```

```
print("Total rows:", df.shape[0])
print("Total columns:", df.shape[1])
```

```
Total rows: 599
Total columns: 10
```

```
df = df.dropna()
df.fillna(df.mean(numeric_only=True), inplace=True)
print(df.isnull().sum())
```

```
TotalSteps      0
TotalDistance    0
VeryActiveDistance  0
ModeratelyActiveDistance  0
LightActiveDistance  0
VeryActiveMinutes  0
FairlyActiveMinutes  0
LightlyActiveMinutes  0
SedentaryMinutes  0
Calories         0
dtype: int64
```

```
numeric_cols = df.select_dtypes(include=np.number)

Q1 = numeric_cols.quantile(0.25)
Q3 = numeric_cols.quantile(0.75)
IQR = Q3 - Q1

df = df[~((numeric_cols < (Q1 - 1.5 * IQR)) |
          (numeric_cols > (Q3 + 1.5 * IQR))).any(axis=1)]

print("Shape after removing outliers:", df.shape)
```

Shape after removing outliers: (494, 10)

```
le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

df.head()
```

	TotalSteps	TotalDistance	VeryActiveDistance	ModeratelyActiveDistance	LightActiveDistance	VeryActiveMinutes	FairlyActiveMinutes
0	13162	8.50	1.88	0.55	6.06	25	
1	10735	6.97	1.57	0.69	4.71	21	
2	10460	6.74	2.44	0.40	3.91	30	
3	9762	6.28	2.14	1.26	2.83	29	
4	12669	8.16	2.71	0.41	5.04	36	

```
print(df.shape)
```

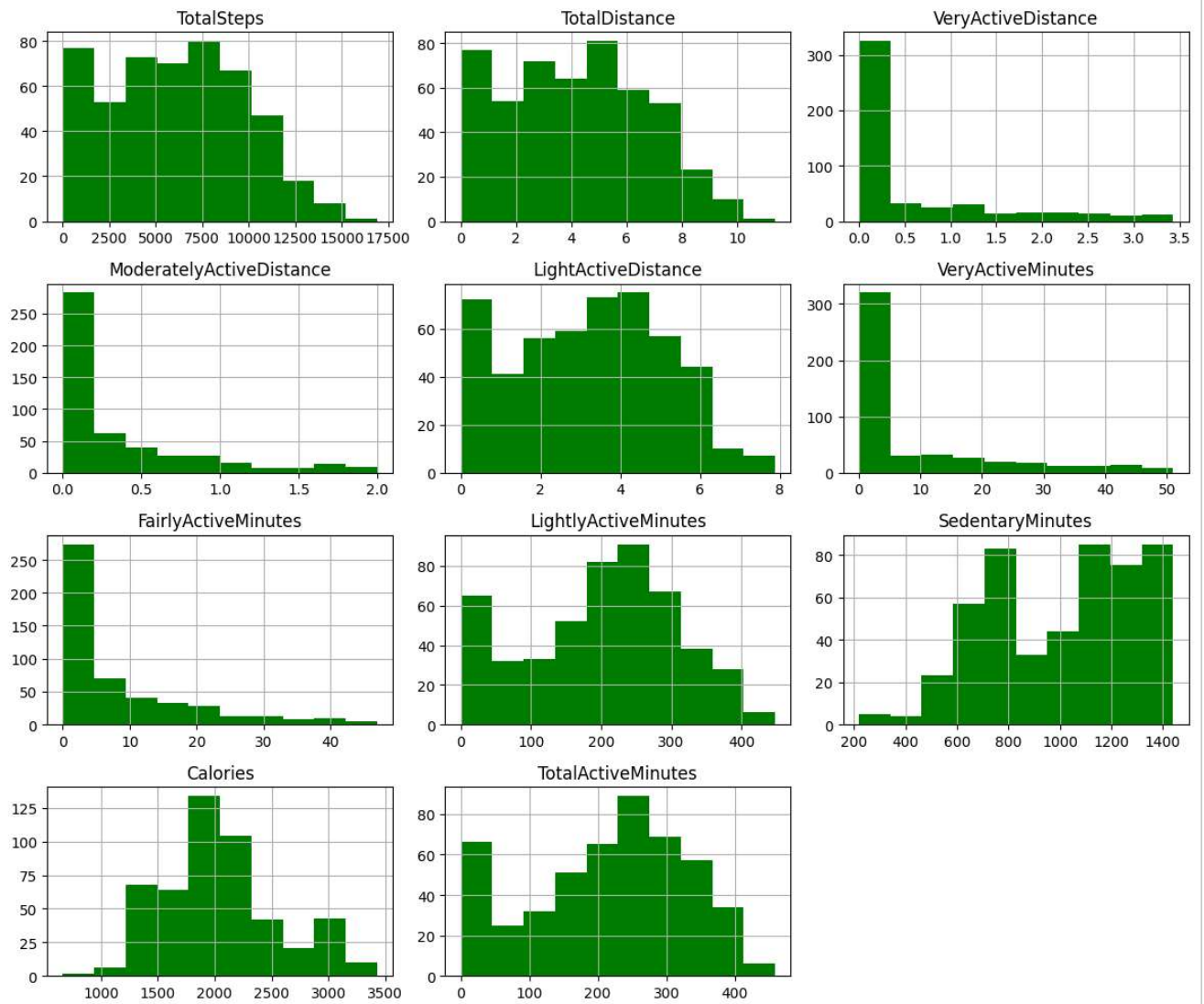
(494, 10)

```
df['TotalActiveMinutes'] = (
    df['VeryActiveMinutes'] +
    df['FairlyActiveMinutes'] +
    df['LightlyActiveMinutes']
)

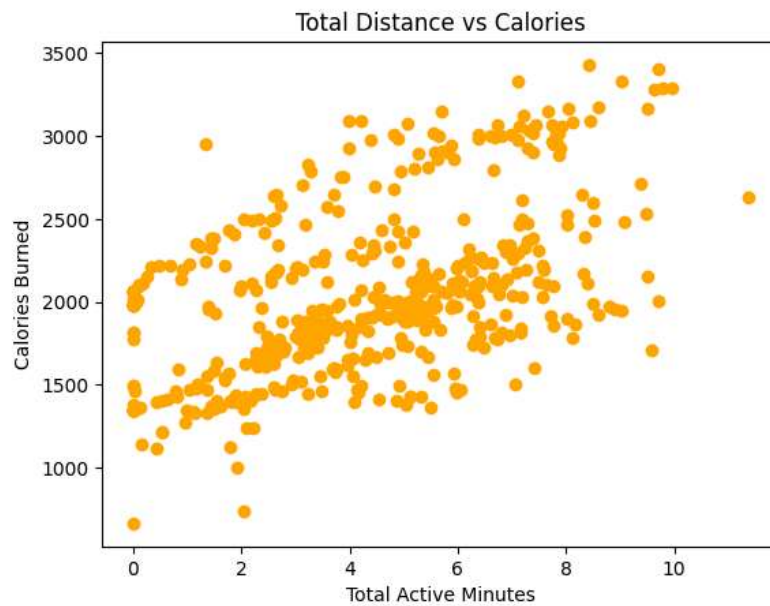
df.head()
```

	TotalSteps	TotalDistance	VeryActiveDistance	ModeratelyActiveDistance	LightActiveDistance	VeryActiveMinutes	FairlyActiveMinutes	TotalActiveMinutes
0	13162	8.50	1.88	0.55	6.06	25		51
1	10735	6.97	1.57	0.69	4.71	21		47
2	10460	6.74	2.44	0.40	3.91	30		64
3	9762	6.28	2.14	1.26	2.83	29		61
4	12669	8.16	2.71	0.41	5.04	36		74

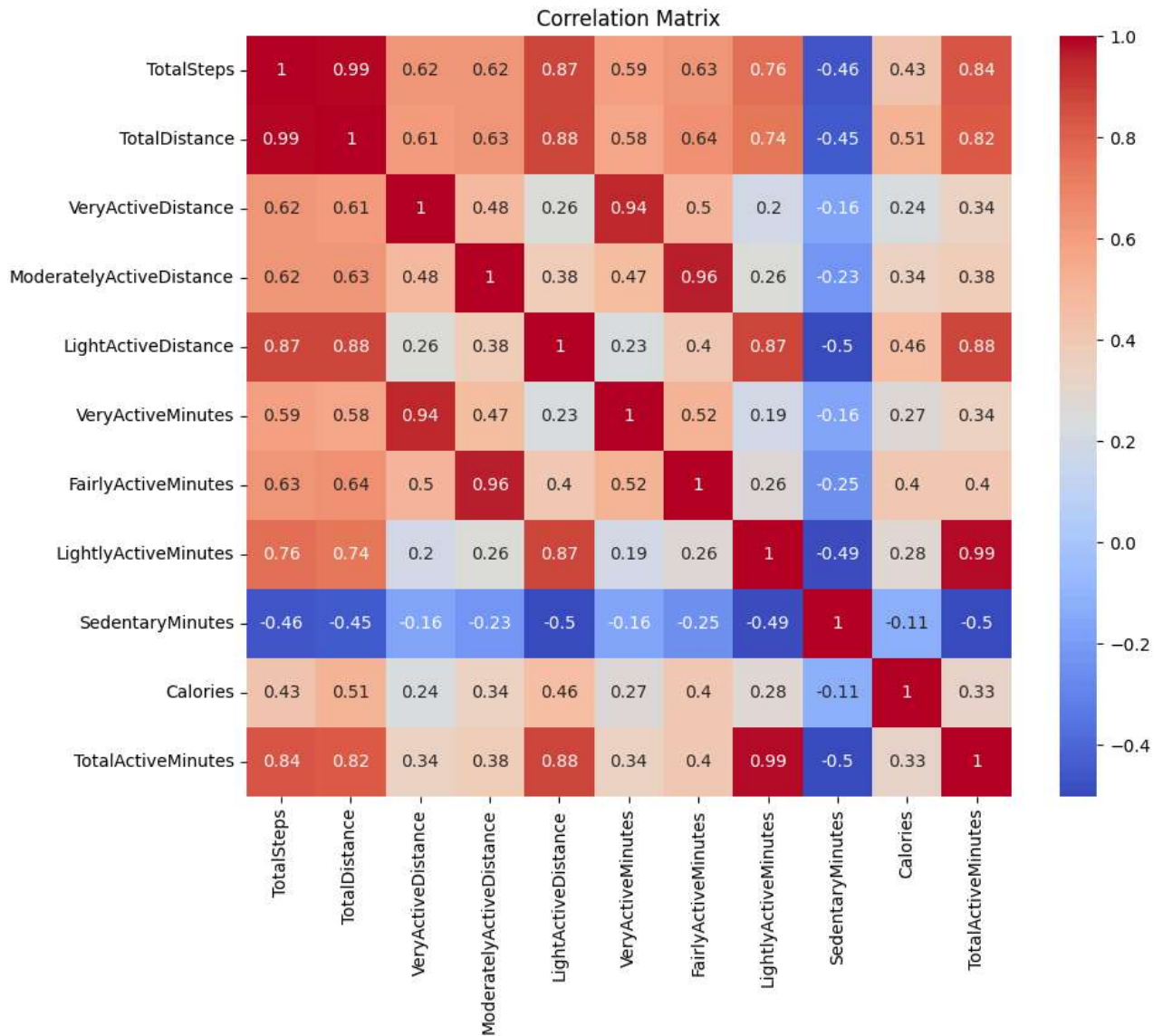
```
df.hist(figsize=(12,10), color="Green")
plt.tight_layout()
plt.show()
```



```
plt.scatter(df['TotalDistance'], df['Calories'], color="Orange")
plt.xlabel("Total Active Minutes")
plt.ylabel("Calories Burned")
plt.title("Total Distance vs Calories")
plt.show()
```



```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



```
# Import library
from sklearn.model_selection import train_test_split

# Define Features (X) and Target (y)
X = df.drop('Calories', axis=1) # All columns except target
y = df['Calories'] # Target column

# Split dataset (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Check shapes
print("Training Features Shape:", X_train.shape)
print("Testing Features Shape:", X_test.shape)
print("Training Target Shape:", y_train.shape)
print("Testing Target Shape:", y_test.shape)
```

```
Training Features Shape: (395, 10)
Testing Features Shape: (99, 10)
Training Target Shape: (395,)
Testing Target Shape: (99,)
```

```
# Import required libraries
import numpy as np
import pandas as pd
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Select features directly (no new column created)
df = pd.read_csv("dailyActivity_merged.csv")
df = df.dropna()
X = df[['TotalSteps', 'TotalDistance', 'VeryActiveMinutes', 'SedentaryMinutes']]
y = df['Calories']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# ----- USER INPUT -----
print("\nEnter values to predict Calories:\n")

steps = float(input("Total Steps: "))
distance = float(input("Total Distance: "))
very_active = float(input("Very Active Minutes: "))
sedentary = float(input("Sedentary Minutes: "))

user_input = pd.DataFrame({
    'TotalSteps': [steps],
    'TotalDistance': [distance],
    'VeryActiveMinutes': [very_active],
    'SedentaryMinutes': [sedentary]
})

prediction = model.predict(user_input)

prediction = model.predict(user_input)

print("\nPredicted Calories Burned:", prediction[0])

```

Enter values to predict Calories:

Total Steps: 7000
 Total Distance: 5.8
 Very Active Minutes: 35
 Sedentary Minutes: 451

Predicted Calories Burned: 2905.223159823582

```

# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# Select features
X = df[['TotalSteps', 'TotalDistance', 'VeryActiveMinutes', 'SedentaryMinutes']]
y = df['Calories']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Random Forest model
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# ----- USER INPUT -----
print("\nEnter values to predict Calories (Random Forest):\n")

steps = float(input("Total Steps: "))
distance = float(input("Total Distance: "))
very_active = float(input("Very Active Minutes: "))
sedentary = float(input("Sedentary Minutes: "))

```

```
# Create DataFrame with proper column names
user_input = pd.DataFrame({
    'TotalSteps': [steps],
    'TotalDistance': [distance],
    'VeryActiveMinutes': [very_active],
    'SedentaryMinutes': [sedentary]
})

# Predict
prediction = rf_model.predict(user_input)

print("\nPredicted Calories Burned (Random Forest):", prediction[0])
```

Enter values to predict Calories (Random Forest):

Total Steps: 5000
 Total Distance: 5.2
 Very Active Minutes: 25
 Sedentary Minutes: 114

Predicted Calories Burned (Random Forest): 1849.78

```
# Import libraries
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

# Select features
X = df[['TotalSteps', 'TotalDistance', 'VeryActiveMinutes', 'SedentaryMinutes']]
y = df['Calories']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Linear Regression
# -----
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)

lr_r2 = r2_score(y_test, lr_pred)
lr_mse = mean_squared_error(y_test, lr_pred)

# -----
# Random Forest
# -----
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

rf_r2 = r2_score(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)

# -----
# Print Results
# -----
print("📊 Linear Regression Results:")
print("R² Score:", lr_r2)
print("MSE:", lr_mse)

print("\n📊 Random Forest Results:")
print("R² Score:", rf_r2)
print("MSE:", rf_mse)

# Comparison
print("\n🔍 Model Comparison:")
if rf_r2 > lr_r2:
    print("Random Forest has higher R² (better).")
else:
```